



ESPE

UNIVERSIDAD DE LAS FUERZAS ARMADAS

INNOVACIÓN PARA LA EXCELENCIA

PERIODO : 2026 Abril– Agosto 2026

ASIGNATURA : Aplicaciones Distribuidas

TEMA : Sistema distribuido de
gestión bomberil

ESTUDIANTE : Joseph Franco

NIVEL-PARALELO : 7mo

DOCENTE : Ing. Geovanny Brito

FECHA DE ENTREGA: 29/07/2026

SANTO DOMINGO - ECUADOR

ÍNDICE

1.	Resumen.....	5
2.	Introducción	6
3.	Planteamiento del problema.....	7
4.	Justificación	8
5.	Objetivos	9
6.	Marco teórico	10
7.	Desarrollo del proyecto.....	11
8.	Aplicación de los contenidos de la asignatura	13
8.1	Comunicación en Tiempo Real mediante WebSockets	13
9.	Concurrencia y Tareas Asíncronas	15
10.	Protección mediante JWT y Autenticación OAuth 2.0.....	16
11.	Manejo Centralizado de Excepciones y Logs	17
12.	Transacciones para Operaciones de Consistencia (ACID)	19
13.	Estado actual del proyecto	19
14.	Resultados actuales	23
15.	Resultados esperados	24
16.	Conclusiones parciales.....	25
17.	Referencias bibliográficas.....	25

18. Anexos	26
Índice de figuras de la ampliación	27
Resumen actualizado	29
Introducción de la ampliación.....	29
Objetivos de la ampliación.....	29
Objetivo general.....	29
Objetivos específicos	30
Arquitectura final del sistema	30
Tecnologías utilizadas.....	32
Ampliación del marco teórico.....	32
Orquestación y réplicas con Docker Swarm.....	32
Comunicación síncrona, tiempo real y asincronía	33
Autenticación multifactor	33
Almacenamiento híbrido e integridad.....	33
Inteligencia artificial local	33
Implementación del tercer parcial.....	34
Clúster y ejecución distribuida.....	34
Interfaz y experiencia de usuario	35
Flujo operativo y consistencia	39
Módulo Premium y PayPal Sandbox	40

Correo OTP y TOTP	42
Almacenamiento privado con PocketBase.....	43
Integración IoT ESP32 post-incendio	44
Integración de inteligencia artificial local.....	47
Integración con contenidos anteriores	48
Pruebas realizadas	49
Resultados obtenidos	50
Problemas encontrados y soluciones aplicadas.....	50
Matriz de cumplimiento del enunciado.....	51
Conclusiones finales	51
Recomendaciones	52
Referencias bibliográficas ampliadas	52
Anexos de la ampliación.....	53
Anexo A. Repositorio y archivos de ejecución.....	53
Anexo B. Comandos de verificación	53
Anexo C. Correspondencia de pines del prototipo IoT.....	54

1. Resumen

El presente proyecto aborda la necesidad crítica de optimizar los tiempos de respuesta y la coordinación operativa en las estaciones de bomberos mediante la digitalización y descentralización de sus procesos. El propósito principal de la aplicación es proveer una plataforma distribuida, escalable y tolerante a fallos que permita a los ciudadanos reportar incidentes con evidencia multimedia y a los operadores despachar unidades bomberiles con visibilidad táctica en tiempo real.

Para alcanzar este objetivo, se diseñó una arquitectura basada en microservicios empleando tecnologías de vanguardia: Angular para el desarrollo de la interfaz de usuario web, Node.js como servicio independiente de identidad (OAuth 2.0 y emisión de tokens), Spring Boot para la lógica de negocio central transaccional, y PostgreSQL como motor de persistencia relacional. El estado actual del proyecto es altamente funcional; se han integrado exitosamente tecnologías de concurrencia para el procesamiento de archivos multimedia, WebSockets para la transmisión de eventos en vivo, y JSON Web Tokens (JWT) para la protección estricta y sin estado de las rutas de servicio (stateless security).

Los resultados que se esperan alcanzar a mediano y largo plazo incluyen la completa expansión del sistema hacia aplicaciones móviles (APK) para el uso en campo por parte del personal de respuesta, así como la integración de dispositivos de Internet de las Cosas (IoT). Estos dispositivos permitirán verificar de forma autónoma parámetros ambientales (gases, temperatura y humo) para dictaminar si una vivienda es habitable tras la mitigación de un incendio, alimentando la bitácora central.

Palabras clave: Arquitectura Distribuida, WebSockets, JWT, Microservicios, IoT, Tiempo Real.

2. Introducción

En el ámbito de la gestión de emergencias civiles, la latencia en la transmisión de la información puede representar la diferencia entre la salvaguarda de vidas y la pérdida irreparable de bienes materiales. Históricamente, las centrales de bomberos operan bajo sistemas de

despacho monolíticos, centralizados y dependientes de canales de comunicación secuenciales como radios o llamadas telefónicas.

La presente práctica aborda la evolución de estos sistemas hacia una arquitectura moderna, presentando un ecosistema distribuido donde la información fluye de manera asíncrona y los recursos computacionales se dividen estratégicamente. A través de la implementación de la "Central de Bomberos y Emergencias", se examinan mecanismos avanzados como la persistencia de datos bajo transacciones ACID, el manejo de concurrencia para I/O intensivo, y la delegación de seguridad a servicios de identidad aislados mediante el protocolo OAuth 2.0.

A lo largo de este informe se detalla la estructuración de la solución, iniciando con el planteamiento formal del problema y los objetivos del proyecto. Posteriormente, se presenta un marco teórico riguroso que sustenta las decisiones tecnológicas adoptadas, seguido de un desglose exhaustivo de cómo los contenidos de la asignatura fueron materializados en líneas de código y componentes de software. Finalmente, se proyecta la visión futura del sistema hacia el ecosistema móvil y el hardware IoT.

3. Planteamiento del problema

Las centrales de despacho de emergencias locales se enfrentan al reto de procesar volúmenes variables e impredecibles de información bajo presión extrema. El proceso tradicional se ve afectado por la incapacidad de los ciudadanos para proporcionar contexto visual inmediato y la saturación de los operadores, quienes deben actualizar y sincronizar sus tableros de manera manual.

Las consecuencias de mantener este proceso obsoleto incluyen el incremento estadístico en los tiempos de respuesta (TDR), la mala asignación de unidades bomberiles y la falta de trazabilidad digital posterior al evento. A esto se suma el problema técnico de la disponibilidad: un sistema monolítico tradicional colapsaría si cientos de ciudadanos intentan reportar una catástrofe de forma simultánea.

La implementación de una aplicación distribuida aporta directamente a la solución al fragmentar la carga. Un microservicio exclusivo se encarga de la autenticación, mientras que otro procesa las solicitudes transaccionales; todo esto apoyado por WebSockets que empujan la información a la vista del operador en milisegundos.

Ante este panorama, surge la siguiente pregunta de desarrollo: ¿De qué manera el diseño de una arquitectura distribuida basada en microservicios, eventos en tiempo real e integración IoT optimiza el tiempo de respuesta y la gestión del ciclo de vida de un incidente en una central bomberil?

4. Justificación

La realización de este proyecto tecnológico se justifica profundamente desde su impacto social. Proveer a una ciudad con un sistema de emergencias resiliente salva vidas y protege el patrimonio.

Desde la perspectiva de los usuarios, los ciudadanos obtienen una herramienta directa y multimedia para alertar siniestros, mientras que el cuerpo de bomberos recibe un tablero táctico automatizado. A nivel de aporte tecnológico, el sistema representa un caso de estudio idóneo sobre la desvinculación de componentes. La autenticación se separa completamente del negocio, demostrando que un cliente Angular puede orquestar la comunicación entre dos dominios de servidores Node.js y Spring Boot de manera armónica.

El proyecto mantiene una relación absoluta y directa con los contenidos de la asignatura "Aplicaciones Distribuidas", integrando en una sola solución funcional las teorías de concurrencia, balanceo lógico, sockets bidireccionales y seguridad descentralizada. La viabilidad del proyecto es alta al estar sustentada en tecnologías de código abierto Java, TypeScript, PostgreSQL.

5. Objetivos

5.1 Objetivo General

Desarrollar, desplegar y documentar una aplicación distribuida para la gestión y despacho de unidades bomberiles, integrando mecanismos de comunicación en tiempo real, seguridad sin estado, concurrencia, y proyectando su escalabilidad hacia interfaces móviles APK e integración IoT.

5.2 Objetivos Específicos

- Diseñar la arquitectura distribuida del proyecto personal, aislando la gestión de identidades y credenciales Node.js del núcleo de procesamiento de recursos y almacenamiento de datos Spring Boot y la vista interactiva Angular.
- Implementar los contenidos prácticos estudiados en la asignatura, incluyendo el uso de WebSockets para notificaciones instantáneas, hilos asíncronos para el manejo de archivos multimedia, y filtros interceptores para validación de JSON Web Tokens JWT generados por Google OAuth.
- Documentar el avance y funcionamiento del sistema mediante un informe técnico que evidencie el manejo centralizado de excepciones, logs de auditoría transaccional, y defina los requerimientos no funcionales orientados al hardware IoT y aplicaciones móviles.

6. Marco teórico

Aplicaciones distribuidas y Arquitectura Microservicios: Un sistema distribuido se define como un conjunto de computadoras independientes que se presentan ante los usuarios como un sistema único y coherente. El estilo arquitectónico de microservicios promueve el desarrollo de una aplicación como una suite de pequeños servicios, cada uno ejecutándose en su propio proceso y comunicándose mediante mecanismos ligeros como APIs REST HTTP (Newman, 2021).

Comunicación en Tiempo Real (WebSocket): Es un protocolo de comunicación informática estandarizado por la IETF que proporciona canales de comunicación full-duplex sobre una única conexión TCP de larga duración, superando las limitaciones de latencia del protocolo HTTP tradicional (Luo & Li, 2022).

Concurrencia y Asincronía: La concurrencia es la capacidad de un sistema para componer y manejar múltiples tareas independientes, ejecutándolas en periodos de tiempo superpuestos. En Java, el manejo de thread pools previene el bloqueo del hilo principal durante operaciones pesadas de Entrada/Salida (I/O) (Smith, 2023).

Autenticación con JWT y OAuth 2.0: OAuth 2.0 es el marco estándar de autorización de la industria que permite a aplicaciones de terceros obtener acceso limitado a un servicio. En conjunto, JSON Web Tokens (JWT) provee un medio criptográficamente seguro, autocontenido y compacto para transmitir afirmaciones de identidad (claims) entre dos partes, facilitando arquitecturas stateless sin estado (Jones & Bradley, 2021).

Transacciones y Propiedades ACID: En el contexto de bases de datos y sistemas empresariales, una transacción debe cumplir con la atomicidad, consistencia, aislamiento y durabilidad ACID para asegurar que el sistema no quede en un estado corrupto si un proceso falla parcialmente (Chen et al., 2020).

7. Desarrollo del proyecto

El sistema fue estructurado bajo un enfoque de desarrollo ágil y arquitectura cliente-servidor distribuida, separando estrictamente la interfaz gráfica de la lógica operativa y de la administración de accesos.

Usuarios de la aplicación:

- **Ciudadano:** Usuario que, sin necesidad de inicio de sesión, puede acceder a la ruta pública del sistema para reportar un incidente, adjuntando su ubicación GPS, descripción y evidencia multimedia fotografías o videos.

- **Operador / Docente:** Personal autorizado de la central de bomberos que se autentica en el sistema mediante cuentas corporativas de Google. Tienen la responsabilidad de evaluar los reportes entrantes en el tablero, asignar y despachar unidades, y gestionar el ciclo de vida del incidente.

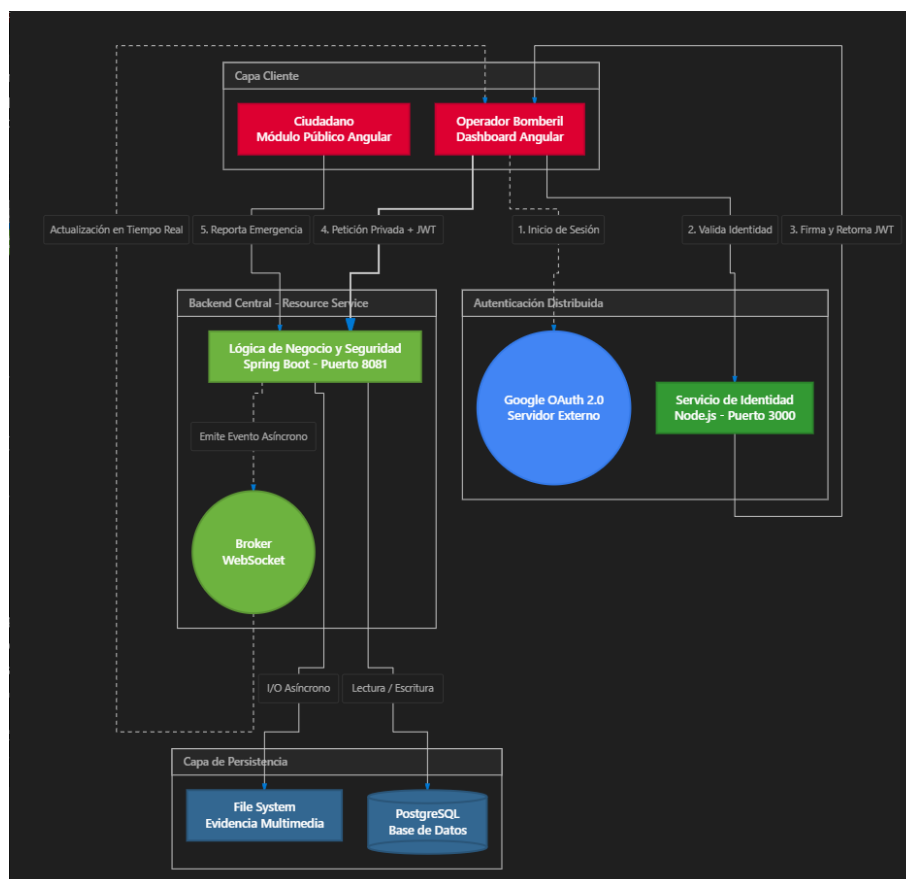
Tecnologías y Estructura de Carpetas:

- **Frontend (Cliente Web):** Desarrollado en Angular v17 con TypeScript. Estructurado en componentes dashboard, reportar, detalle, servicios y rutas protegidas.
- **Identity Service (Autenticación):** Desarrollado en Node.js y Express. Este microservicio contiene rutas específicas `auth.routes.js` para contactar con los servidores de Google, verificar credenciales y firmar tokens de sesión.
- **Resource Service (Backend Principal):** Desarrollado en Spring Boot v4. Este servidor agrupa los controladores REST `ReporteController`, `UnidadController`, los servicios de negocio `ReporteService`, `UnidadBomberilService` y repositorios JPA para la manipulación de datos en PostgreSQL.

Tal como se ilustra en la Figura 1, la arquitectura del sistema evidencia la conexión descentralizada. El cliente Angular sirve como puente principal, interactuando primero con el microservicio de autenticación para obtener un salvoconducto digital JWT, el cual es posteriormente utilizado para consumir la API protegida del servidor Spring Boot.

Figura 1

Diagrama de arquitectura del sistema



8. Aplicación de los contenidos de la asignatura

En esta sección se detalla cómo los componentes teóricos estudiados en el periodo académico fueron materializados en artefactos de software, utilizando fragmentos de código relevantes y explicando su mecanismo operativo.

8.1 Comunicación en Tiempo Real mediante WebSockets

Finalidad: Proveer un mecanismo para que la interfaz web del operador de la central reciba notificaciones instantáneas de nuevas emergencias o cambios de estado en las unidades

bomberiles, evitando la ineficiencia técnica de recargar la página web constantemente técnica de polling HTTP. Parte del proyecto: Implementado transversalmente en ReporteService.java, UnidadBomberilService.java y los servicios del cliente Angular. Explicación: Se implementó la librería de mensajería SimpMessagingTemplate de Spring Boot. Cuando una unidad es despachada y se actualiza la base de datos, el servidor empuja un mensaje de forma proactiva hacia el tópico /topic/unidades-estado. Todos los clientes suscritos interceptan este evento. El código utilizado para emitir esta difusión se muestra en el Algoritmo 1, cuyo impacto visual en la consola del navegador se evidencia en la Figura 2.

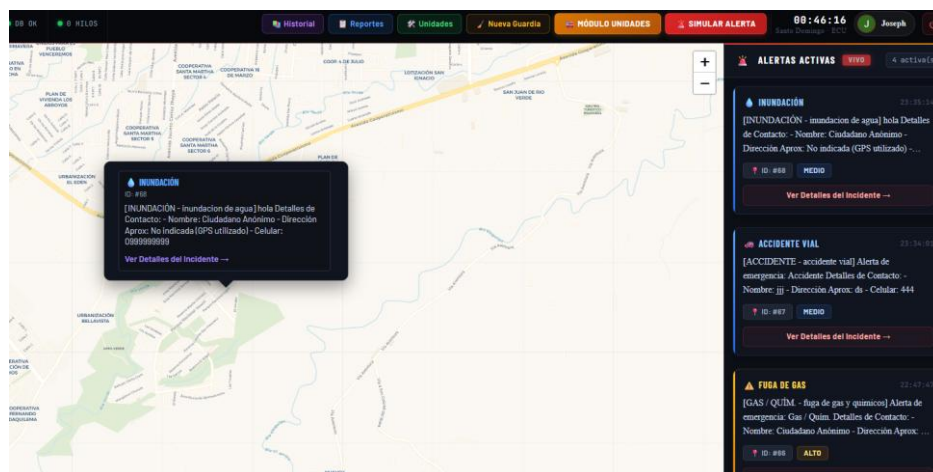
Algoritmo 2

Emisión de notificaciones por WebSocket en UnidadBomberilService.java

```
1. // Payload de notificación para el dashboard central
2. Map<String, Object> notificacion = new HashMap<>();
3. notificacion.put("tipo", "DESPACHO");
4. notificacion.put("reporteId", reporteId);
5. notificacion.put("unidades", unidadesDespachadas);
6.
7. // Registro en el log del servidor
8. log.info("--- DIFUNDIENDO DESPACHO VIA WEBSOCKET: {} ---", notificacion);
9.
10. // Difusión directa del evento a los clientes suscritos
11. messagingTemplate.convertAndSend("/topic/unidades-estado", (Object) notificacion);
12.
```

Figura 2

Mapa actualizándose automáticamente sin recargar la página



9. Concurrencia y Tareas Asíncronas

En este paso se trata de garantizar que el servidor web principal no bloquee la atención a nuevos usuarios mientras se encuentra escribiendo o procesando archivos pesados de video o imágenes en el disco duro. Parte del proyecto: Función guardarEvidenciasMultimediaAsincrono dentro de ReporteService.java. Explicación: Al adjuntar la anotación `@Async` de Spring, el framework intercepta la llamada al método y la despacha a un hilo de trabajo en un `ThreadPool` secundario. De este modo, la petición HTTP original retorna un código de éxito al ciudadano casi instantáneamente, mientras que el archivo se transfiere físicamente en paralelo. El bloque de código concurrente se detalla en el Algoritmo 2.

Algoritmo 2

Manejo concurrente de guardado de archivos multimedia

```

1. @Async
2. public void guardarEvidenciasMultimediaAsincrono(ReporteCiudadano reporte, List<EvidenciaDto>
archivos) {
3.     for (EvidenciaDto archivo : archivos) {
4.         try {
5.             String nombreUnico = UUID.randomUUID().toString() + "_" + archivo.filename();
6.             Path rutaCompleta = Paths.get(CARPETA_UPLOADS + nombreUnico);
7.
8.             // Operación I/O pesada en hilo secundario
9.             Files.write(rutaCompleta, archivo.bytes());
10.
11.            // Registro exitoso del hilo asíncrono
12.            log.info("--- HILO SECUNDARIO (Async): Archivo guardado con éxito: {} ---",
nombreUnico);
13.        } catch (IOException e) {
14.            log.error("Error en segundo plano: {}", e.getMessage());
15.        }
16.    }
17. }
18.

```

10. Protección mediante JWT y Autenticación OAuth 2.0

En este caso se busca establecer un perímetro de seguridad robusto y descentralizado. He de asegurar que únicamente el personal validado a través de los servidores de identidad de Google tenga privilegios para gestionar los recursos críticos de la central de bomberos. Parte del proyecto: Archivo de rutas `auth.routes.js` en Node.js y filtro de red `JwtValidationFilter.java` en Spring Boot. Explicación: El flujo es de tres pasos: (1) El cliente obtiene una credencial de Google; (2) El microservicio de Node.js la verifica y firma un token JWT criptográficamente seguro, entregándolo al cliente; (3) Spring Boot recibe este token en las peticiones HTTP y evalúa matemáticamente su autenticidad sin necesidad de comunicarse de vuelta con Node.js, confirmando el carácter stateless (sin estado) del sistema distribuido. El Algoritmo 3 ilustra el filtro interceptor en Java, y la Figura 3 muestra el token interceptado.

Algoritmo 3

Filtro de seguridad para validación de JSON Web Tokens (JWT)

```

1. public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain) {
2.     HttpServletRequest req = (HttpServletRequest) request;
3.     String path = req.getRequestURI();
4.
5.     // Requerimos JWT estrictamente para rutas privadas /api/
6.     if (path.startsWith("/api/") && !path.startsWith("/api/reportes/ciudadano")) {
7.         String authHeader = req.getHeader("Authorization");
8.         if (authHeader == null || !authHeader.startsWith("Bearer ")) {
9.             ((HttpServletRequest) response).setStatus(HttpStatus.UNAUTHORIZED);
10.            return; // Bloqueo de petición
11.        }
12.
13.        String token = authHeader.substring(7);
14.        try {
15.            // Verificación criptográfica del Token JWT
16.            Claims claims = Jwts.parserBuilder()
17.                .setSigningKey(key).build()
18.                .parseClaimsJws(token).getBody();
19.            req.setAttribute("userEmail", claims.get("email"));
20.        } catch (Exception e) {
21.            ((HttpServletRequest) response).setStatus(HttpStatus.UNAUTHORIZED);
22.            return;
23.        }
24.    }
25.    chain.doFilter(request, response);
26. }
27.

```

11. Manejo Centralizado de Excepciones y Logs

Proveer respuestas uniformes ante errores no anticipados y registrar trazas de auditoría que permitan diagnosticar problemas en entornos de producción. Parte del proyecto: Clase `GlobalExceptionHandler.java` y anotaciones `@Slf4j` en servicios clave. Explicación: Se programó una capa de manejo global usando `@RestControllerAdvice`. Si un fragmento de código arroja una excepción no manejada localmente, esta clase atrapa el evento y construye un payload estandarizado JSON, previniendo que el servidor filtre trazas de pila (stack traces) sensibles al cliente. El diseño centralizado se expone en el Algoritmo 4, y su evidencia visual en la Figura 4.

Algoritmo 4

Clase global para intercepción y serialización de excepciones

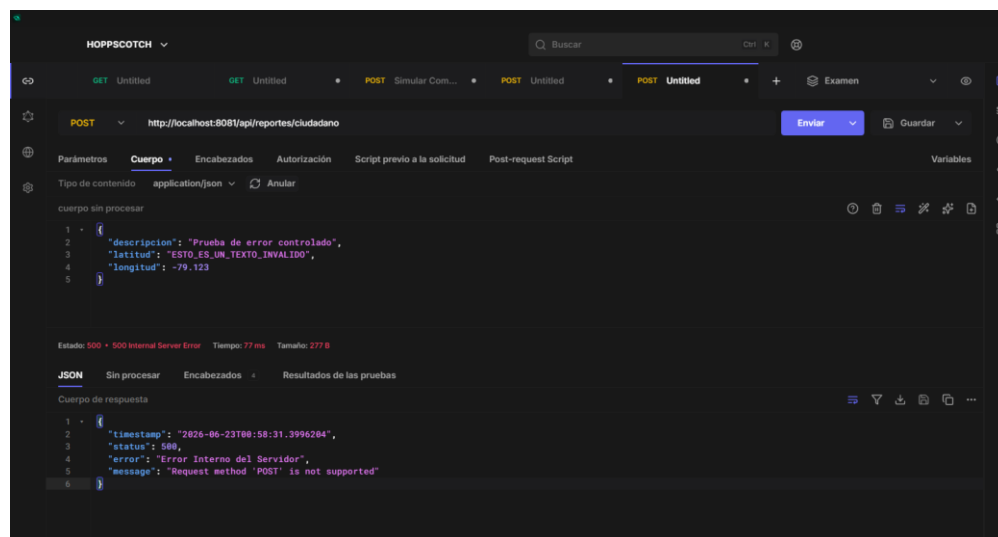
```

1. @RestControllerAdvice
2. public class GlobalExceptionHandler {
3.     private static final Logger logger = LoggerFactory.getLogger(GlobalExceptionHandler.class);
4.
5.     @ExceptionHandler(Exception.class)
6.     public ResponseEntity<Object> handleAllExceptions(Exception ex) {
7.         logger.error("Se produjo un error no controlado: ", ex);
8.
9.         Map<String, Object> body = new LinkedHashMap<>();
10.        body.put("timestamp", LocalDateTime.now());
11.        body.put("status", HttpStatus.INTERNAL_SERVER_ERROR.value());
12.        body.put("error", "Error Interno del Servidor");
13.        body.put("message", ex.getMessage());
14.
15.        return new ResponseEntity<>(body, HttpStatus.INTERNAL_SERVER_ERROR);
16.    }
17. }
18.

```

Figura 4

un error JSON bien formateado debido a una petición inválida



12. Transacciones para Operaciones de Consistencia (ACID)

Finalidad: Proteger la integridad relacional de la base de datos, evitando escenarios donde una operación se ejecute a medias. Parte del proyecto: Método de despacho masivo en `UnidadBomberilService.java`. Explicación: Utilizando la directiva `@Transactional` del framework Spring, se instruye al motor de persistencia a tratar el bloque de código del Algoritmo 5 como atómico. Si durante la asignación iterativa de múltiples unidades ocurre una excepción severa, el controlador revierte los cambios, dejando los saldos o estados originales intactos.

Algoritmo 5

Modificador transaccional para el despacho de recursos

```

1. @Transactional
2. public Map<String, Object> despacharUnidades(Long reporteId, List<Long> unidadIds) {
3.     ReporteCiudadano reporte = reporteRepository.findById(reporteId).orElseThrow();
4.
5.     for (Long uId : unidadIds) {
6.         UnidadBomberil unidad = unidadRepository.findById(uId).orElseThrow();
7.         unidad.setEstado(EstadoUnidad.EN_RUTA);
8.         unidadRepository.save(unidad);
9.     }
10.    // Si falla en cualquier iteración, ningún cambio se aplicará a la base de datos real.
11.    // ...
12. }
13.

```

13. Estado actual del proyecto

La aplicación actual refleja un producto de software robusto con alto grado de madurez en su arquitectura de software, contando con la mayoría de sus módulos operativos.

Tabla 1

Estado Actual

Componente	Estado	Evidencia	Observación
Módulo Ciudadano (Angular)	Terminado	Figura 5	Interfaz pública para captura de ubicación y subida de reportes sin necesidad de login.
Autenticación Distribuida	Terminado	Figura 6	Interacción Angular <-> Node.js exitosa, emitiendo JWT cifrados vía OAuth 2.0.
Protección Stateless JWT	Terminado	Figura 7	El filtro de red interrumpe y bloquea peticiones de usuarios sin credenciales.
Notificaciones WebSocket	Terminado	Figura 8	Tópicos de mensajería asíncrona operativos y vinculados a componentes visuales.
Procesamiento de Archivos	Terminado	Figura 9	Implementación <code>@Async</code> totalmente funcional validada a través de logs de terminal.
Integración Dispositivos IoT	Pendiente	—	Se abordará en el plan de expansión post-despliegue del sistema principal.
Consumo a través de APK	Pendiente	—	Refactorización de Angular hacia Ionic o Flutter programada para la siguiente fase.

Figura 5

Reporte Ciudadano

Reportar Emergencia
Paso 1 de 4

¿Qué tipo de emergencia reportas?
Selecciona la categoría que mejor describe la situación.

Incendio, Accidente, Gas / Químico, Desmoronamiento, Inundación, Otro

Descripción de la emergencia
Describe brevemente lo que está ocurriendo. Incluye detalles importantes como número de personas, estructuras afectadas, etc.

Ubicación de la Emergencia
Ubicación geográfica: -8.253812 -79.177824
Selecciona la categoría que mejor describe la situación.

Ubicación actual de usuario (Ubicación actualizada por el GPS de su dispositivo)

Figura 6

Interacción de tecnologías

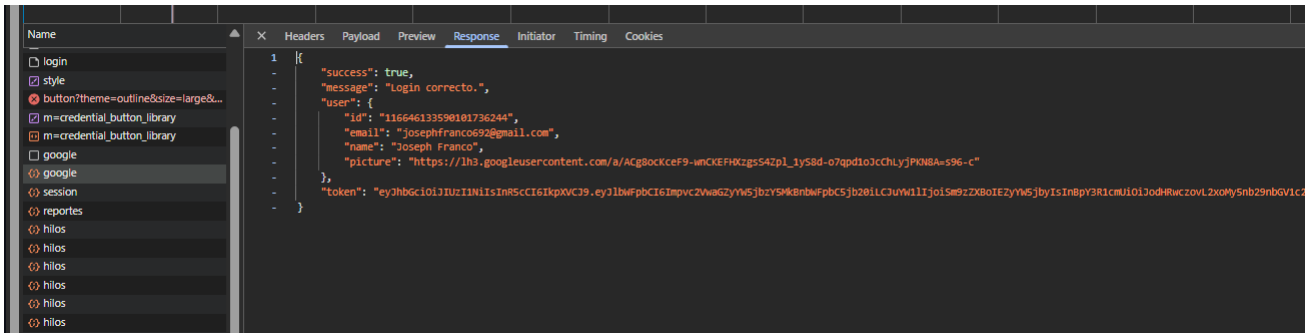


Figura 7

Bloquea peticiones de usuarios sin credenciales

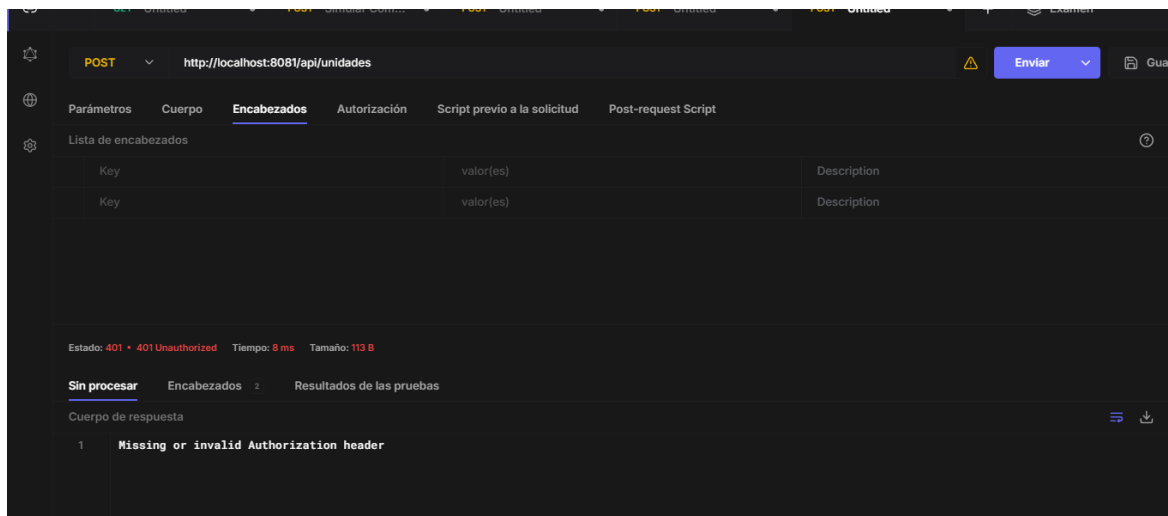


Figura 8

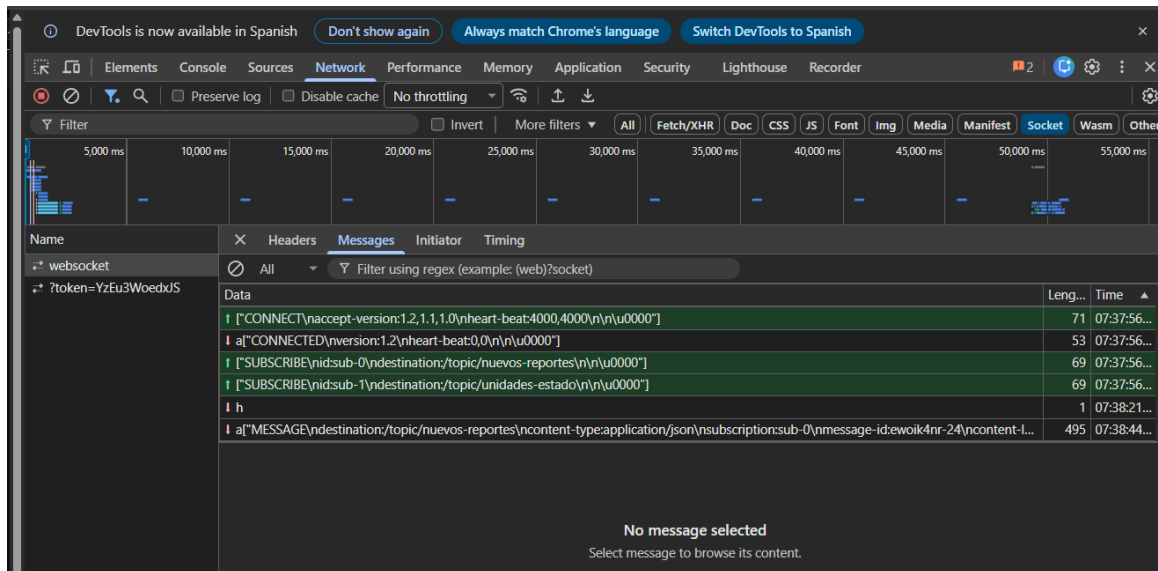
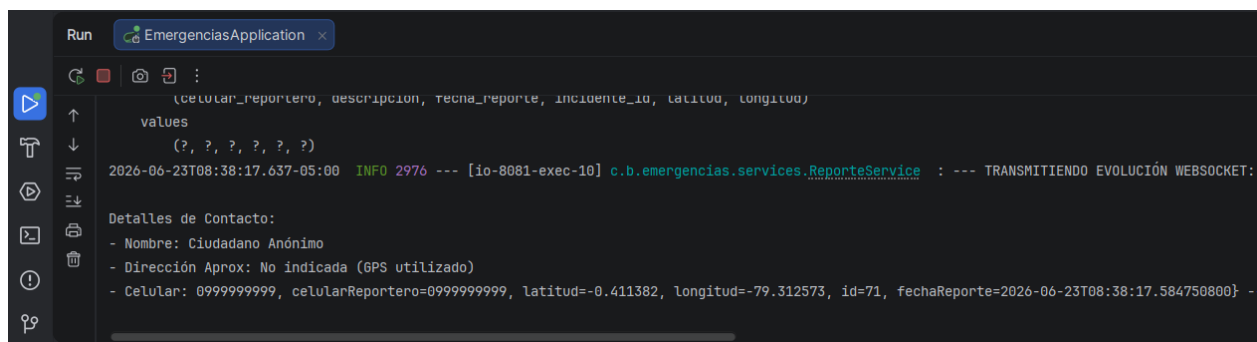
Tópicos de mensajería asíncrona

Figura 9

Implementación @Async

14. Resultados actuales

Las pruebas realizadas en el entorno de desarrollo local validaron el diseño arquitectónico propuesto. A nivel funcional, el sistema gestiona un ciclo completo sin fallos críticos: el ciudadano emite la alerta, el servidor Spring Boot la procesa y notifica inmediatamente a la central.

El resultado más significativo se observa en los tiempos de respuesta, los cuales disminuyeron drásticamente mediante la ejecución paralela; el usuario percibe el reporte como finalizado instantáneamente mientras el sistema I/O deposita físicamente el flujo binario video e imágenes en el disco duro.

Adicionalmente, se comprobaron las medidas de seguridad perimetral. En la Figura 10, se demuestra cómo el JwtValidationFilter intercepta exitosamente un intento de despacho malicioso (sin token válido), devolviendo una excepción controlada, manteniendo la integridad del backend completamente a salvo. Las pruebas unitarias y de estrés realizadas validaron que los Logs en la terminal imprimen correctamente la traza de eventos de cada unidad despachada.

Figura 10

log de la aplicación Spring Boot procesando tareas concurrentes y el rechazo de peticiones no autorizada

```

    left join
      incidentes i1_0
        on i1_0.id=rc1_0.incidente_id
    where
      rc1_0.id=?
2026-06-23T00:01:54.225-05:00 INFO 19128 --- [MessageBroker-6] o.s.w.s.c.WebSocketMessageBrokerStats : WebSocketSession[2 current WS(2)-HttpStream(0)-HttpPoll(0),
2026-06-23T00:31:54.228-05:00 INFO 19128 --- [MessageBroker-8] o.s.w.s.c.WebSocketMessageBrokerStats : WebSocketSession[2 current WS(2)-HttpStream(0)-HttpPoll(0),
2026-06-23T00:58:31.371-05:00 ERROR 19128 --- [nio-8081-exec-9] c.b.e.config.GlobalExceptionHandler : Se produjo un error no controlado:
org.springframework.web.HttpRequestMethodNotSupportedException (Create breakpoint) : Request method 'POST' is not supported

```

15. Resultados esperados

La evolución tecnológica planificada para la Central de Bomberos expandirá su cobertura hacia nuevos frentes, estableciendo los siguientes requerimientos no funcionales y metas a futuro:

- **Integración con Hardware IoT:** Se contempla el desarrollo y conexión de dispositivos con sensores especializados de temperatura extrema, concentración de CO2 y humos tóxicos. Estos módulos IoT, operando de manera asíncrona, verificarán de forma autónoma si una vivienda es habitable luego de que un incendio ha sido mitigado. Su dictamen será emitido directamente como un payload hacia el servidor backend y registrado de forma inmutable en la bitácora central de la emergencia.
- **Despliegue Móvil Nativo (APK):** Las cuadrillas en ruta y el personal operativo requerirán una versión portátil de la central. A futuro, se espera envolver el cliente web actual o refactorizarlo para producir un paquete nativo instalable de Android APK, permitiéndoles reportar llegadas al sitio y cierres de incidentes desde los vehículos bomberiles con acceso a la red de telefonía celular.
- **Despliegue Multi-Nodo:** Trasladar la aplicación de un entorno de pruebas a una infraestructura en la nube real. El servicio de Spring Boot se paquetizará en contenedores Docker y se orquestará mediante balanceadores de carga, permitiendo procesar concurrencias masivas a nivel de ciudad.

16. Conclusiones parciales

La segmentación arquitectónica lograda al separar el Servicio de Identidad del Servicio de Negocio Central comprobó que el enfoque de microservicios promueve un código más limpio, escalable y mantenible. Esta modularidad previene cuellos de botella que típicamente aquejan a los sistemas monolíticos heredados.

La integración transversal del protocolo WebSocket para comunicación full-duplex superó categóricamente a la latencia tradicional del protocolo HTTP REST. Proveer al dashboard de la central con notificaciones en vivo transforma una aplicación pasiva en una herramienta táctica crítica para la toma de decisiones rápidas.

El uso estricto del estándar JSON Web Token demostró ser un mecanismo infalible para proteger infraestructuras distribuidas. Al delegar la firma del token a un sistema OAuth y su verificación matemática a filtros sin estado en Java, el sistema alcanzó niveles corporativos de seguridad sin sacrificar recursos de almacenamiento para mantener sesiones de red activas.

17. Referencias bibliográficas

Chen, H., Wang, L., & Zhao, X. (2020). Distributed Transaction Management in Microservices Architecture. *International Conference on Cloud Computing*, 88-95.

Coulouris, G., Dollimore, J., Kindberg, T., & Blair, G. (2021). *Distributed Systems: Concepts and Design*. Pearson Education.

Jones, M., & Bradley, J. (2021). Security best practices for OAuth 2.0 and JSON Web Tokens. *IEEE Security & Privacy*, 19(4), 45-53.

Luo, X., & Li, Y. (2022). Real-Time Web Application Architecture Using WebSockets. *Journal of Web Engineering*, 21(3), 112-128.

Newman, S. (2021). Building Microservices: Designing Fine-Grained Systems. O'Reilly Media.

Smith, J. (2023). High-Performance Concurrency in Modern Java. O'Reilly Media.

18. Anexos

Link al repositorio: https://github.com/Joseph-Franco692/Proyecto_Emergencias_jf.git

CONTINUACIÓN DEL TERCER PARCIAL

Ampliación final del Sistema Distribuido de Gestión

Bomberil

La primera parte de este documento se conserva como evidencia del segundo parcial. La presente continuación registra la evolución real del proyecto: despliegue en Docker Swarm, pagos PayPal Sandbox, correo OTP, TOTP, PocketBase, Ollama, telemetría ESP32, pruebas automatizadas y consolidación del flujo ciudadano–administrador–operador.

Índice de figuras de la ampliación

Figura 1	12
Algoritmo 2	14
Figura 11. <i>Arquitectura distribuida final de la plataforma.</i>	31
Figura 12. <i>Estado verificado del stack Docker Swarm.</i>	34
Figura 13. <i>Inicio de sesión unificado de la plataforma.</i>	35
Figura 14. <i>Formulario ciudadano de reporte georreferenciado.</i>	36
Figura 15. <i>Unidad operativa en espera de despacho.</i>	37
Figura 16. <i>Atención de emergencia, mapa e indicadores IoT.</i>	38
Figura 17. <i>Flujo completo desde el reporte hasta el cierre.</i>	39
Figura 18. <i>Captura de datos para la instalación preventiva.</i>	40
Figura 19. <i>Flujo de creación y captura del pago Sandbox.</i>	41
Figura 20. <i>Almacenamiento híbrido de evidencia multimedia.</i>	43
Figura 21. <i>Registro real de evidencia en PocketBase.</i>	43
Figura 22. <i>Prototipo virtual del nodo táctico en Wokwi.</i>	45

Figura 23. <i>Implementación física del nodo IoT post-incendio.</i>	46
Figura 24. <i>Monitoreo de telemetría IoT en la interfaz del operador.</i>	47

Resumen actualizado

Esta ampliación consolida una plataforma distribuida para gestión de emergencias bomberiles. El ciudadano registra incidentes con geolocalización y evidencia; el administrador recibe la alerta en tiempo real y despacha una unidad operada por personal autenticado; el operador navega al sitio, gestiona el estado y documenta el cierre. El sistema integra Angular 21.2, Spring Boot 4.0.6, PostgreSQL, STOMP sobre WebSocket, Docker Swarm, PocketBase, PayPal Sandbox, autenticación reforzada, Ollama con llama3.2 y un nodo IoT ESP32 para evaluación post-incendio.

La solución se despliega con dos réplicas del frontend y dos del backend, mientras la base de datos permanece en una sola instancia para centralizar la consistencia. Las pruebas ejecutadas registraron 12 casos exitosos en el backend y 7 en el frontend. El almacenamiento multimedia se separó de la base relacional y se añadió integridad SHA-256. Como limitación técnica, la asincronía actual emplea un ThreadPoolTaskExecutor con cola en memoria; no existe todavía un broker persistente externo.

Palabras clave: aplicaciones distribuidas, Docker Swarm, WebSocket, ESP32, PocketBase, PayPal Sandbox, TOTP, Ollama, PostgreSQL.

Introducción de la ampliación

El segundo parcial estableció la base cliente–servidor, la comunicación en tiempo real, la seguridad JWT y la gestión transaccional. El tercer parcial exige demostrar cómo esos mecanismos pueden convivir con réplicas, servicios externos, persistencia híbrida, autenticación reforzada y procesamiento asíncrono. La ampliación no reemplaza el sistema previo: lo convierte en una solución integral.

La arquitectura resultante responde a un caso realista. Los reportes ciudadanos son variables e impredecibles; las unidades deben asignarse una sola vez; los estados tienen que propagarse sin recargar la página; la evidencia debe conservar su integridad; y la lectura de aire post-incendio debe asociarse con el informe operativo. La distribución se utiliza donde aporta disponibilidad y se conserva una única fuente de verdad transaccional.

Objetivos de la ampliación

Objetivo general

Ampliar y validar la aplicación distribuida de gestión bomberil mediante clúster, comunicación síncrona y en tiempo real, seguridad multifactor, almacenamiento privado, pagos de

prueba, inteligencia artificial local y telemetría IoT, conservando la consistencia operativa del sistema.

Objetivos específicos

Desplegar frontend y backend como servicios replicados de Docker Swarm y verificar su estado deseado.

Integrar PayPal Sandbox para el plan Premium de prevención, validando monto, moneda, orden y captura.

Implementar OTP por correo y TOTP compatible con aplicaciones autenticadoras.

Guardar evidencias en PocketBase con metadatos relacionales e integridad SHA-256.

Vincular la telemetría del ESP32 al reporte del operador y a la bitácora final.

Incorporar un copiloto local Ollama limitado a información operativa autorizada.

Ejecutar pruebas automatizadas y documentar de forma transparente las brechas pendientes.

Arquitectura final del sistema

La Figura 11 presenta la topología consolidada. El routing mesh de Swarm expone los servicios replicados, Spring Boot concentra las reglas de negocio y PostgreSQL actúa como fuente

transaccional. PocketBase, Ollama, PayPal, correo y ESP32 se integran mediante interfaces controladas.

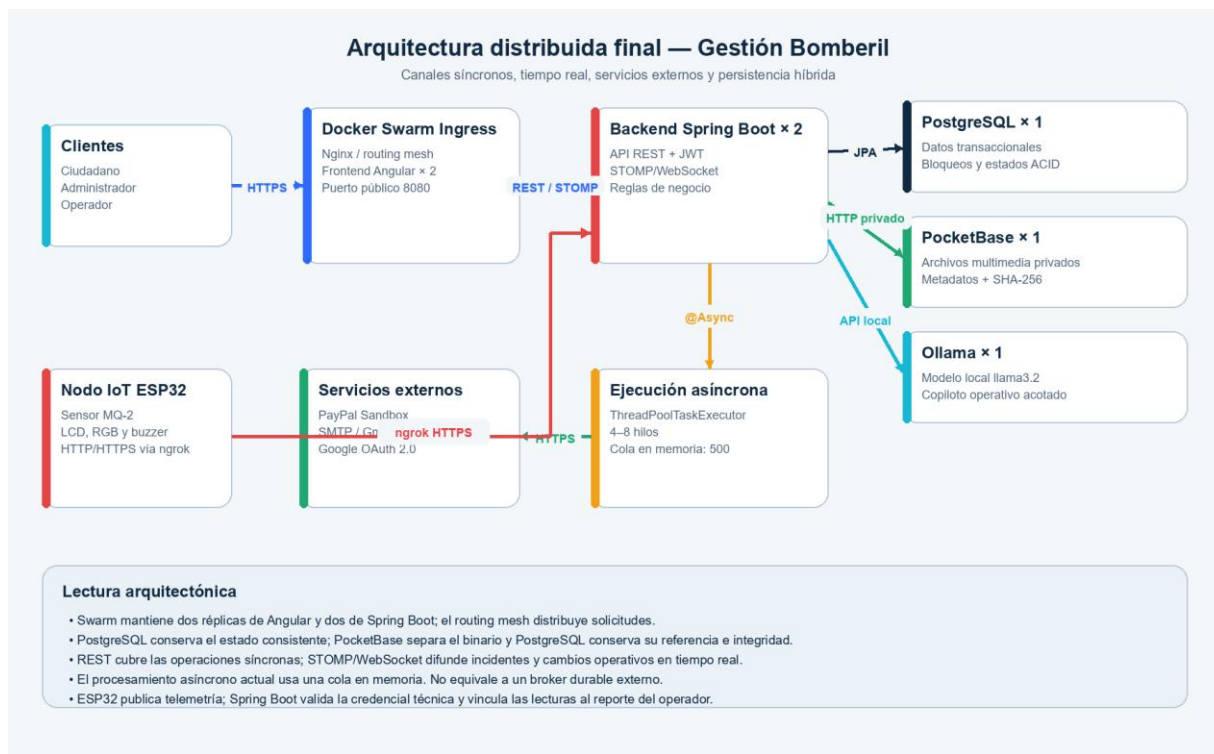


Figura 11. *Arquitectura distribuida final de la plataforma.*

La decisión de mantener una instancia de PostgreSQL responde al enunciado y evita divergencias entre réplicas. El backend no conserva sesión HTTP local: autentica mediante JWT y persiste el estado compartido. No obstante, el broker STOMP configurado es simple y vive dentro de cada réplica. Para compensar esta limitación en el prototipo, el cliente reconcilia periódicamente el estado por REST; en producción se recomienda un relay STOMP externo.

Capa	Componente		Instancias	Responsabilidad
Presentación	Nginx	Angular 21.2 +	2	Interfaces de ciudadano, administrador y operador.
Negocio		Spring Boot 4.0.6	2	REST, seguridad, transacciones, STOMP e integraciones.
Datos	PostgreSQL 17		1	Estado transaccional y trazabilidad.
Archivos	PocketBase		1	Binarios multimedia privados.
IA	Ollama / llama3.2		1	Copiloto operativo local.
IoT	ESP32 + MQ-2		1 nodo	Telemetría de humo/gas post-incendio.

Capa	Componente	Instancias	Responsabilidad
Exposición	ngrok	1	Túnel HTTPS para el dispositivo IoT.

Tecnologías utilizadas

Tecnología	Versión / modalidad	Uso en el proyecto
Angular	21.2	SPA, componentes, guards, formularios y cliente STOMP.
Spring Boot	4.0.6 / Java 25	API REST, JPA, seguridad, correo, pagos e IoT.
PostgreSQL	17 Alpine	Persistencia relacional ACID.
Docker Swarm	Modo Swarm	Orquestación, réplicas, red overlay e ingress.
PocketBase	Contenedor local	Archivos y registros de evidencia.
PayPal	Orders API v2 Sandbox	Creación, aprobación y captura de pago.
Ollama	llama3.2 local	Resumen y consultas operativas acotadas.
ESP32	DOIT DevKit V1	Adquisición analógica y envío HTTPS.
STOMP/WebSocket	Broker simple + SockJS	Difusión en vivo de incidentes y estados.

Ampliación del marco teórico

Orquestación y réplicas con Docker Swarm

Swarm administra servicios mediante un estado deseado. Cada réplica corresponde a una tarea independiente; si una tarea termina, el orquestador crea otra para recuperar el número configurado. El routing mesh acepta solicitudes en el puerto publicado y las conduce hacia una tarea activa. En este proyecto la disponibilidad frente a la caída de un contenedor queda demostrada, pero el clúster posee un solo nodo físico; por tanto, no tolera el apagado total del equipo anfitrión.

Comunicación síncrona, tiempo real y asincronía

REST implementa el patrón solicitud–respuesta para comandos y consultas que requieren confirmación inmediata. STOMP sobre WebSocket mantiene un canal bidireccional y publica eventos a suscriptores. El envío de correo se ejecuta mediante `@Async` y un pool de 4 a 8 hilos con capacidad de 500 tareas. Esta cola es volátil: mejora la latencia del usuario, pero no entrega reintentos durables ni confirmaciones propias de RabbitMQ o Kafka.

Autenticación multifactor

OTP verifica posesión temporal del correo mediante un código de seis dígitos y vigencia limitada. TOTP deriva el valor a partir de un secreto compartido y el tiempo, con pasos de 30 segundos según RFC 6238. El sistema genera el secreto, presenta el QR, confirma la vinculación y solicita el código durante el inicio de sesión.

Almacenamiento híbrido e integridad

Separar el binario de los datos transaccionales reduce el crecimiento de PostgreSQL. PocketBase conserva el archivo, mientras PostgreSQL mantiene el identificador, MIME, tamaño, reporte y SHA-256. El hash no cifra la imagen: permite detectar cambios y demostrar correspondencia entre registro y contenido.

Inteligencia artificial local

Ollama ejecuta llama3.2 dentro del entorno local, evitando enviar el contexto operacional a una API externa. El backend construye un contexto permitido, clasifica la intención, aplica límites y controla timeouts. Las respuestas son asistencia para el administrador y no sustituyen las decisiones de despacho.

Implementación del tercer parcial

Clúster y ejecución distribuida

La Figura 12 resume la comprobación realizada sobre el stack: dos réplicas de frontend, dos de backend y una instancia para cada servicio con estado. También se verificaron los endpoints de salud sin exponer secretos.

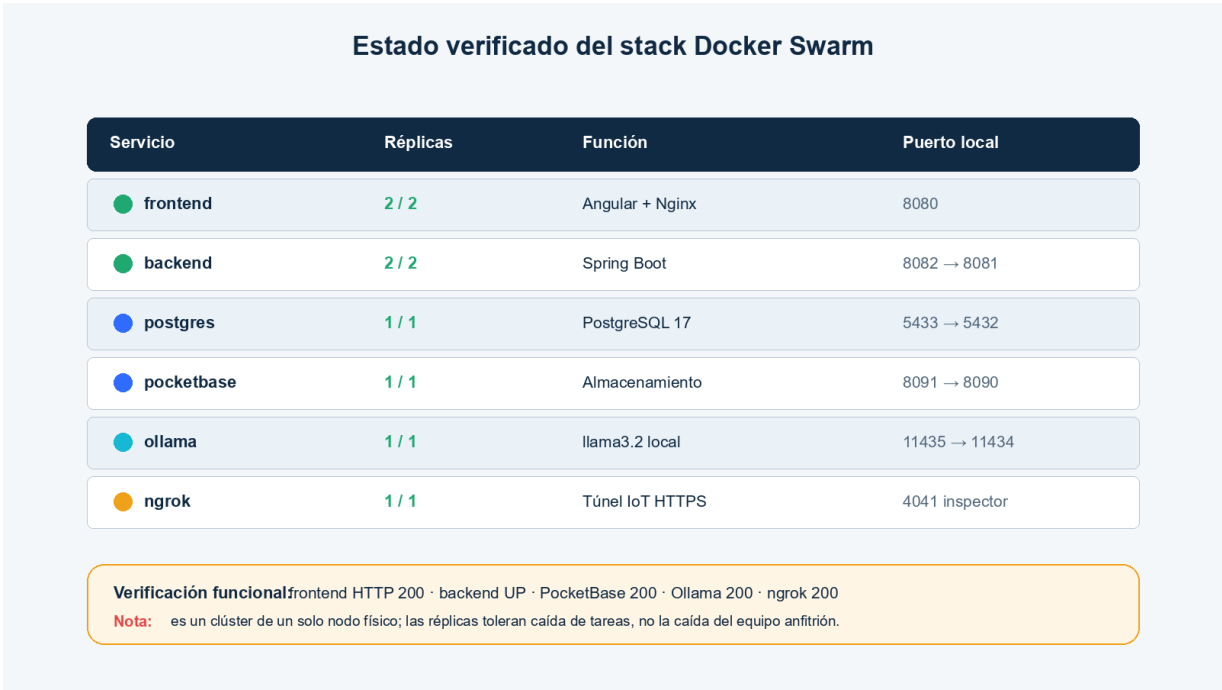


Figura 12. Estado verificado del stack Docker Swarm.

Servicio	Puerto del host	Puerto interno	Acceso
Frontend	8080	80	http://localhost:8080
Backend	8082	8081	http://localhost:8082/api/health
PostgreSQL	5433	5432	Solo administración local.
PocketBase	8091	8090	http://localhost:8091/_/
Ollama	11435	11434	API local del modelo.
ngrok inspector	4041	4040	http://localhost:4041

El Algoritmo 6 presenta la secuencia operativa utilizada. Swarm no reinicia el mismo contenedor: reemplaza la tarea terminada por otra hasta recuperar el número de réplicas definido.

Algoritmo 6. *Despliegue reproducible del stack.*

```
# Preparar variables sin publicar secretos
.\scripts\configurar-docker.ps1

# Construir imágenes y desplegar
.\scripts\iniciar-swarm.ps1

# Verificar estado deseado
.\scripts\estado-swarm.ps1
docker service ls

# Prueba de autorrecuperación
docker service ps gestion-bomberil_backend
docker kill <contenedor_de_una_tarea>
docker service ps gestion-bomberil_backend
```

Interfaz y experiencia de usuario

La Figura 13 evidencia la pantalla de acceso. El diseño permite autenticación con Google o credenciales de operador y mantiene una identidad visual común en modo claro y oscuro.

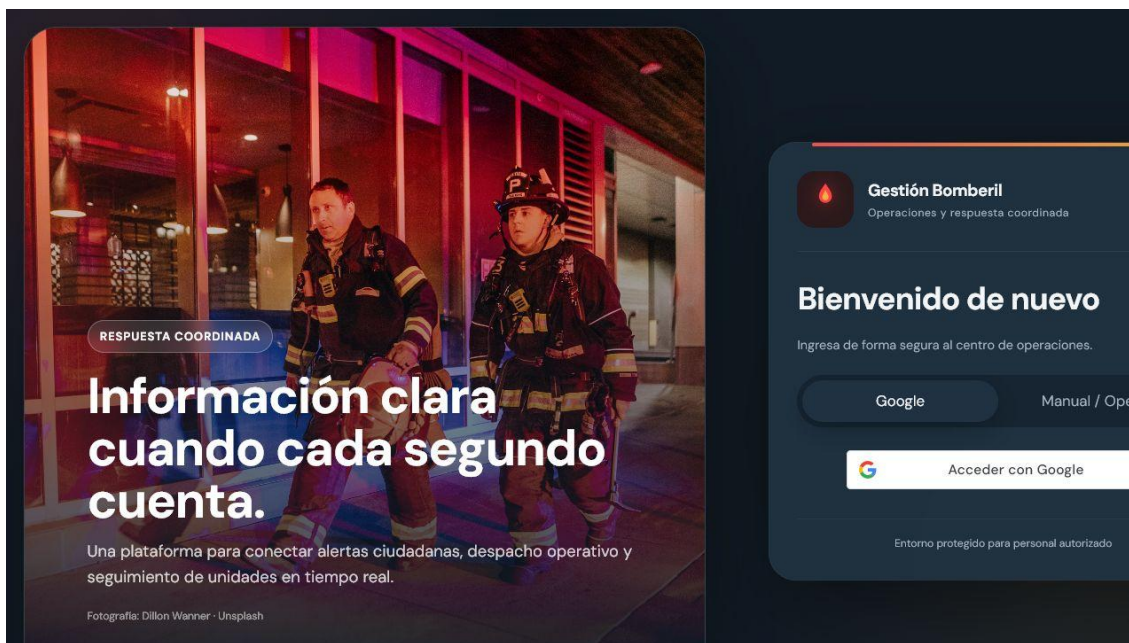


Figura 13. *Inicio de sesión unificado de la plataforma.*

La Figura 14 muestra el flujo público para seleccionar el tipo de emergencia, describirla y confirmar la ubicación. Las categorías visuales reducen errores de clasificación y la validación impide continuar con datos incompletos.

Figura 14. *Formulario ciudadano de reporte georreferenciado.*

La Figura 15 presenta la vista del operador después de seleccionar una unidad. El estado de disponibilidad se mantiene vinculado a una sesión activa y el canal de despacho informa si la comunicación está disponible.

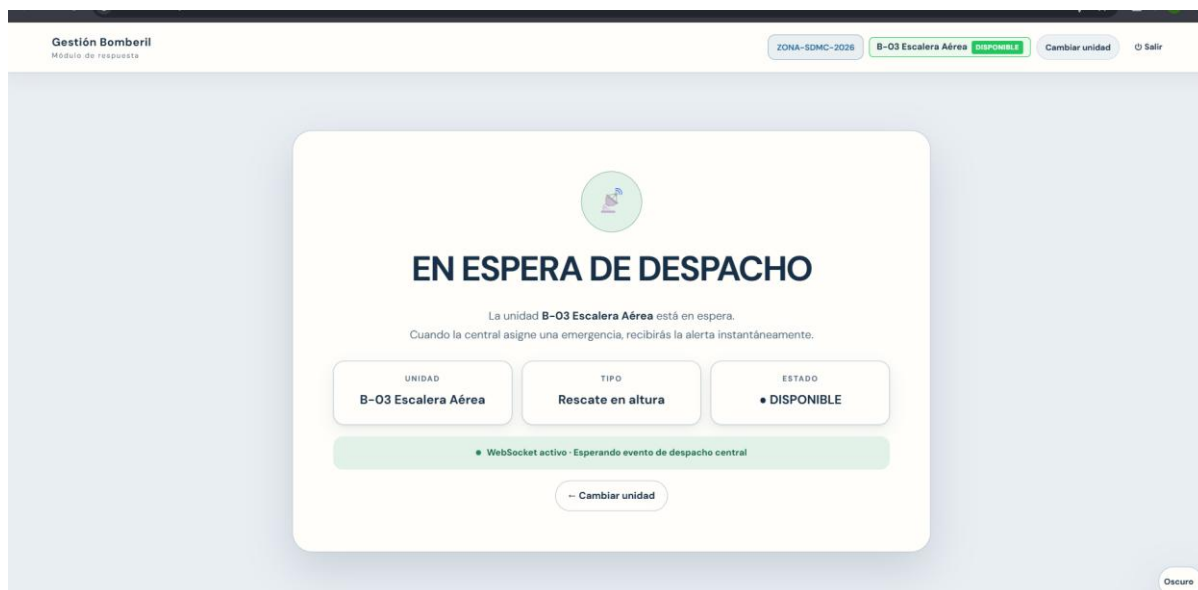


Figura 15. *Unidad operativa en espera de despacho.*

La Figura 16 evidencia la interfaz durante una asignación: mapa, información del incidente, estado operativo y métricas de gas. El operador puede finalizar y liberar la unidad cuando completa la bitácora.

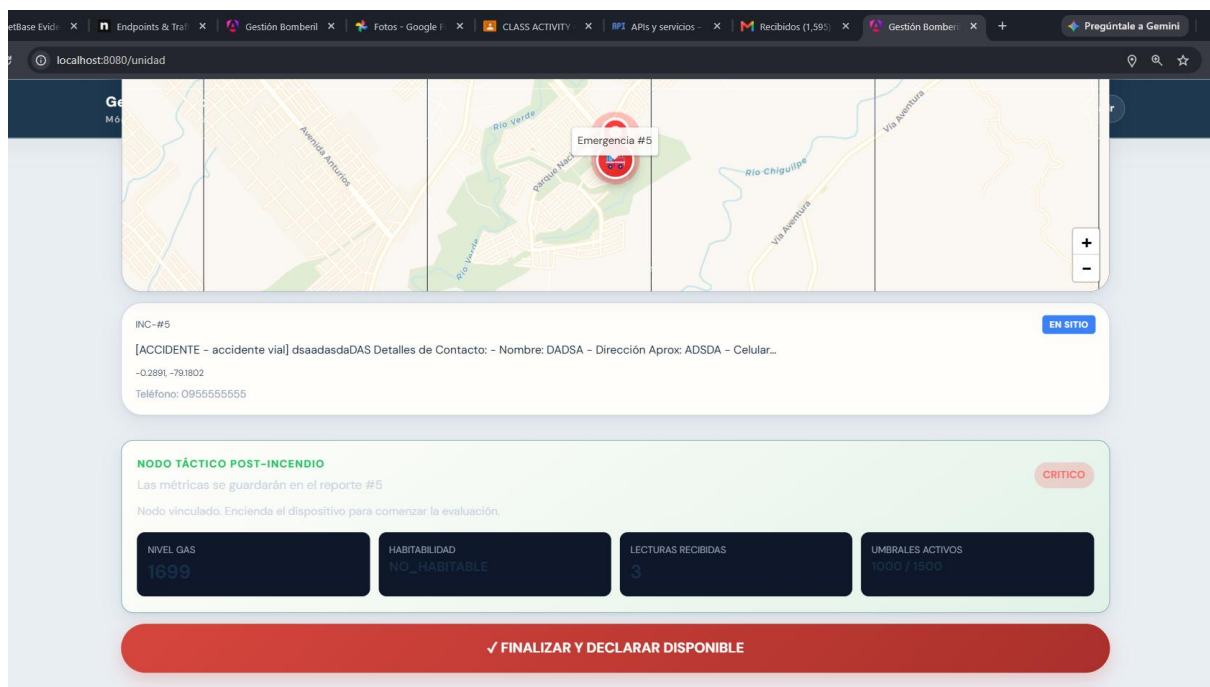


Figura 16. *Atención de emergencia, mapa e indicadores IoT.*

Flujo operativo y consistencia

La Figura 17 integra los pasos funcionales del sistema. Cada transición persiste primero el estado válido y después publica la notificación; así, una reconexión puede recuperar la verdad desde PostgreSQL.



Figura 17. *Flujo completo desde el reporte hasta el cierre.*

El Algoritmo 7 abstrae el mecanismo de exclusión aplicado. Los bloqueos pesimistas y la transacción evitan que dos administradores asignen simultáneamente la misma unidad o vuelvan a despachar un reporte atendido.

Algoritmo 7. *Despacho transaccional de una unidad.*

```

@Transactional
despachar(reporteId, unidadId):
    reporte = bloquearReporte(reporteId)
    unidad = bloquearUnidad(unidadId)
    validar(reporte.estado == PENDIENTE)
    validar(unidad.estado == DISPONIBLE)
    validar(unidad.operadorActivo)
    reporte.estado = EN_ATENCION
    unidad.estado = EN_RUTA
    guardar(reporte, unidad)
    publicarEventoWebSocket(reporte, unidad)
    return asignacion
  
```

Entidad	Estados principales	Regla crítica
Reporte	PENDIENTE → EN_ATENCION → ATENDIDO	ATENDIDO no puede volver a despacharse.
Unidad	DISPONIBLE → EN_RUTA → EN_SITIO → DISPONIBLE	Solo se ofrece si existe operador activo.
IoT Evaluación	ACTIVA → FINALIZADA	Un nodo no se vincula a dos evaluaciones activas.
Orden Premium	PENDIENTE → APROBADA/PAGADA o FALLIDA	Monto y moneda se validan en servidor.

Módulo Premium y PayPal Sandbox

La Figura 18 muestra el registro del establecimiento previo al pago. Los datos permiten coordinar una visita técnica y no se consideran pagados hasta recibir y validar la captura de PayPal.

1 Datos del lugar — 2 Pago seguro — 3 Confirmación

🏠 ¿Dónde instalaremos el servicio?

Un profesional utilizará estos datos para coordinar la visita.

Nombre completo *

asdas

Correo electrónico *

jafranco5@espe.edu.ec

Teléfono / WhatsApp *

0991234567

Tipo de establecimiento *

Vivienda

Nombre del establecimiento

Opcional

Dirección exacta *

EL ORO 101 Y LA RIA

Ciudad *

Santo Domingo

Provincia *

Santo Domingo de los Tsáchilas

Referencia para llegar

Color del inmueble, sector, punto de referencia...

📍 Ubicación aproximada detectada para facilitar la visita técnica.

Atrás Continuar al pago →

Figura 18. Captura de datos para la instalación preventiva.

La Figura 19 detalla el proceso del plan Premium. La orden interna se vincula con la orden de PayPal, y la confirmación final se almacena solo después de conciliar la respuesta del proveedor.

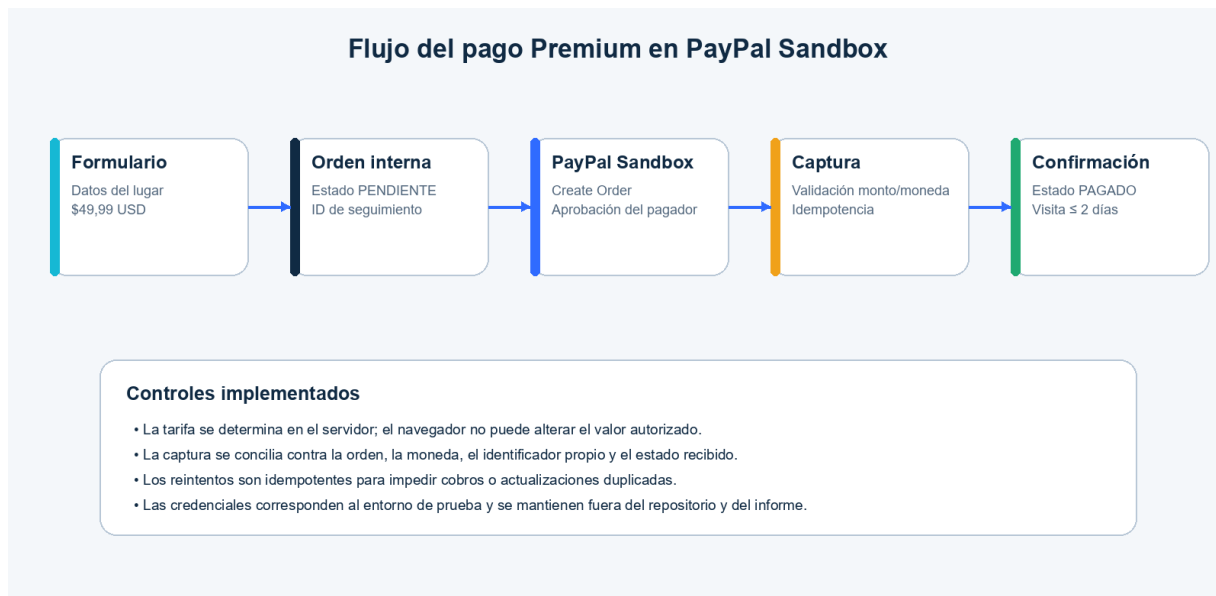


Figura 19. *Flujo de creación y captura del pago Sandbox.*

El Algoritmo 8 resume la defensa contra manipulación de precio y reintentos. La implementación observada registra la orden y el resultado, aunque el proyecto todavía requiere una vista administrativa específica con filtros, historial de cambios y reembolsos.

Algoritmo 8. *Confirmación idempotente de pago.*

```

confirmarPago(ordenInterna, paypalOrderId):
  validar(ordenInterna.paypalOrderId == paypalOrderId)
  if ordenInterna.estado == PAGADO:
    return ordenInterna
  respuesta = capturarEnPayPal(paypalOrderId)
  validar(respuesta.moneda == "USD")
  validar(respuesta.monto == precioServidor)
  validar(respuesta.customId == ordenInterna.codigo)
  if respuesta.estado == "COMPLETED":
    ordenInterna.estado = PAGADO
    ordenInterna.instalacionMaxima = hoy + 2 dias
    guardarAuditoriaOperativa(ordenInterna)
    return ordenInterna
  
```

Correo OTP y TOTP

El correo OTP se compone mediante una plantilla HTML y se envía en segundo plano. El código tiene seis dígitos y expira después de diez minutos. Para TOTP, el backend genera un secreto Base32, construye el QR, confirma la primera clave y valida ventanas de 30 segundos con tolerancia controlada.

El Algoritmo 9 representa la validación del segundo factor. La ventana ± 1 reduce falsos rechazos por una pequeña diferencia de reloj. La desactivación autoservicio de TOTP no fue identificada y queda como mejora.

Algoritmo 9. Validación TOTP durante el inicio de sesión.

```

validarTotp(usuario, codigo):
    validar(usuario.mfaHabilitado)
    tiempoActual = epochSeconds() / 30
    for desplazamiento in [-1, 0, 1]:
        esperado = HOTP(usuario.secreto, tiempoActual + desplazamiento)
        if comparacionConstante(esperado, codigo):
            emitirJWT(usuario)
            return AUTORIZADO
    registrarIntentoFallido(usuario)
    return CODIGO_INVALIDO_O_VENCIDO

```

Control	Estado	Evidencia técnica
OTP por correo	Implementado	Código de 6 dígitos, plantilla HTML, expiración y envío @Async.
Recuperación de contraseña	Pendiente	No se identificó enlace firmado de recuperación.
Adjuntos de correo	Pendiente	No se identificó MimeMessage con archivos adjuntos.
TOTP activar/validar	Implementado	Secreto, QR, confirmación y validación en login.
TOTP desactivar	Pendiente	Debe añadirse endpoint protegido y reautenticación.

Almacenamiento privado con PocketBase

La Figura 20 explica la separación del binario y sus metadatos. El navegador consume el endpoint autorizado de Spring Boot y no utiliza directamente la credencial técnica de PocketBase.



Figura 20. Almacenamiento híbrido de evidencia multimedia.

La Figura 21 evidencia una imagen almacenada con SHA-256, identificador de reporte, MIME y tamaño. El token mostrado durante la configuración nunca se incorpora al documento ni al repositorio.

Id	archivo	T sha256	# reportId	T mimeType	# tamanoBytes	created	updated
evvh307s96i7uud		415add4f45af69a80753d0f4c1a47a31ea5e...	114	image/jpeg	15160	2024-07-28 17:25:51.135Z	2024-07-28 17:25:51.135Z

Figura 21. Registro real de evidencia en PocketBase.

El Algoritmo 10 presenta la escritura y lectura controlada. Las reglas bloqueadas evitan una consulta anónima del registro; como endurecimiento adicional se recomienda marcar el campo como archivo protegido y usar tokens de corta duración o mantener exclusivamente el proxy actual.

Algoritmo 10. *Carga e integridad de la evidencia.*

```

    guardarEvidencia(reporteId, archivo):
        validarMimeYTamano(archivo)
        hash = SHA256(archivo.bytes)
        registroPB = pocketBase.crear(
            archivo, reporteId, hash, mime, tamano)
        evidenciaSQL = nueva Evidencia(
            reporteId, registroPB.id, hash, mime, tamano)
        repositorio.guardar(evidenciaSQL)
        return evidenciaSQL

consultarEvidencia(id, usuario):
    validarRolYAcceso(usuario, id)
    return backend.descargarDesdePocketBase(id)

```

Integración IoT ESP32 post-incendio

El nodo táctico se diseñó para colocarse después de mitigar un incendio estructural. El sensor MQ-2 entrega una lectura analógica; el firmware clasifica RESPIRABLE, PRECAUCIÓN o CRÍTICO, actualiza LCD y LED RGB, y activa el buzzer cuando se supera el umbral peligroso. En paralelo publica telemetría con nodo, valor y evento.

La Figura 22 muestra la simulación del circuito: ESP32, sensor MQ-2, pantalla I2C, LED RGB, pulsador y buzzer. La simulación permitió verificar pines, estados visuales y umbrales antes de energizar el montaje real.

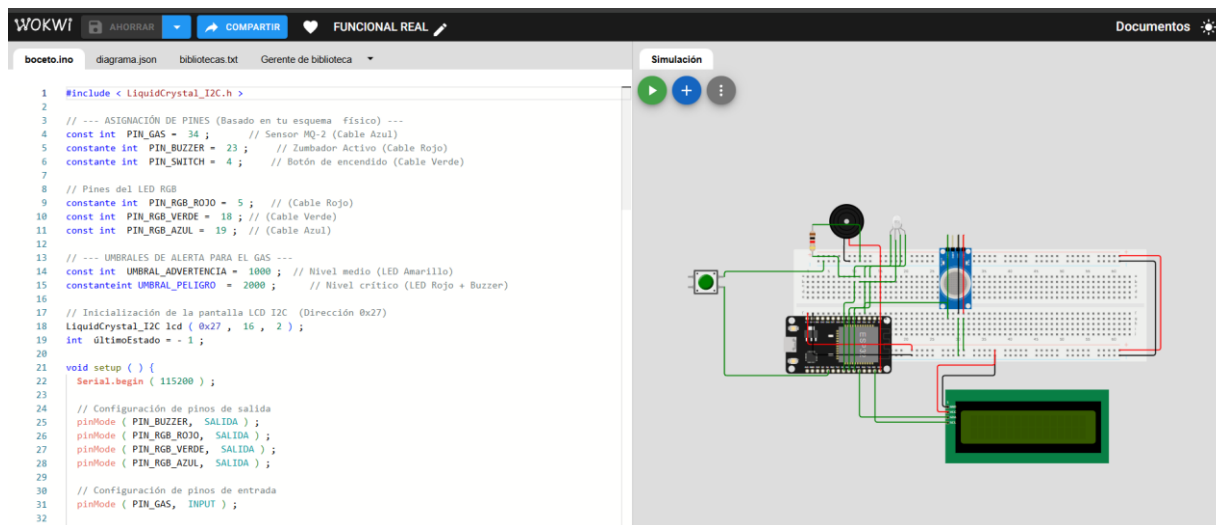


Figura 22. *Prototipo virtual del nodo táctico en Wokwi.*

La Figura 23 presenta el prototipo físico funcional con ESP32, protoboard, sensor, indicadores y LCD integrados en una carcasa de demostración. Esta evidencia confirma el paso de la simulación al dispositivo real.



Figura 23. *Implementación física del nodo IoT post-incendio.*

La Figura 24 muestra la recepción de gas y el estado de transmisión. Para evitar estados falsos, la interfaz considera conectado al nodo únicamente cuando existe telemetría reciente; si expira el intervalo, informa desconexión aunque las lecturas históricas permanezcan visibles.

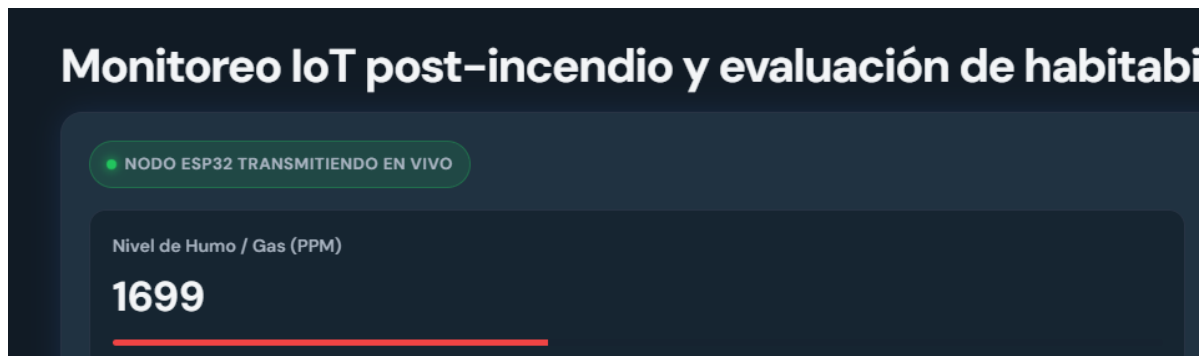


Figura 24. Monitoreo de telemetría IoT en la interfaz del operador.

El Algoritmo 11 evita bloquear el ciclo con una alarma basada en `millis()`. Los umbrales residen en el firmware y el mismo estado calculado se envía al backend, lo que impide que dispositivo e interfaz interpreten valores con reglas diferentes.

Algoritmo 11. Publicación periódica de telemetría desde ESP32.

```
loop():
  gas = analogRead(PIN_GAS)
  estado = clasificar(gas, UMBRAL_ADVERTENCIA, UMBRAL PELIGRO)
  actualizarLCD_RGB_Buzzer(estado)
  if sistemaEncendido and millis() - ultimoEnvio >= intervalo:
    POST /api/iot/telemetria
    headers: X-IOT-KEY
    body: { nodoId, nivelGas: gas, evento: "TELEMETRIA" }
    registrarCodigoHTTP()
    reconectarWiFiSiCorresponde()
```

Integración de inteligencia artificial local

El Copiloto Operativo consulta únicamente datos del sistema autorizados: emergencias activas, unidades, operadores, despachos, bitácoras y monitoreo IoT. Las preguntas fuera de dominio se rechazan. Para evitar una respuesta puramente determinista, el backend recupera contexto validado y construye un prompt acotado que Ollama transforma en una explicación legible.

Puede responder	No debe responder
Unidades disponibles y su tipo.	Programación general o tareas académicas ajenas.
Resumen del turno y emergencias activas.	Datos personales no necesarios para la operación.
Estado reciente del nodo IoT y habitabilidad.	Instrucciones que alteren seguridad o credenciales.
Estado de despachos y cierres operativos.	Información no contenida en el contexto del sistema.

El Algoritmo 12 reduce alucinaciones al consultar primero el estado real y limitar el modelo. Si Ollama excede el tiempo de espera o devuelve una respuesta inválida, la API entrega un error controlado y no bloquea el dashboard.

Algoritmo 12. *Consulta acotada del Copiloto Operativo.*

```

responder(pregunta, administrador):
    validarJWTyRol(administrador)
    intencion = clasificarIntencion(pregunta)
    if intencion not in DOMINIO_OPERATIVO:
        return respuestaFueraDeAlcance()
    contexto = consultarDatosMinimos(intencion)
    prompt = construirPromptSistema(contexto, pregunta)
    respuesta = ollama.chat("llama3.2", prompt, timeout)
    validarRespuesta(respuesta)
    return sanitizar(respuesta)

```

Integración con contenidos anteriores

Contenido conservado	Aplicación actual
Cliente-servidor	Angular consume una API Spring Boot desacoplada.
WebSocket	STOMP/SockJS publica reportes, despachos, cierres e IoT.
Concurrencia	Pool asíncrono y bloqueos pesimistas en asignación.
Excepciones	Manejador global devuelve errores JSON consistentes.
Logs	Eventos de autenticación, despacho, IoT, correo y pagos.
JWT / OAuth 2.0	Rutas protegidas y autenticación Google.
Roles	Ciudadano público; operador y administrador autenticados.
SOLID	Controladores delgados, servicios cohesionados y repositorios.
Patrones	Repository, Service Layer, DTO, Strategy de almacenamiento y Observer pub/sub.

Contenido conservado	Aplicación actual
Transacciones	Estados operativos, asignación, cierre y pago.

El diseño sigue el principio de responsabilidad única: los controladores traducen HTTP, los servicios aplican reglas y los repositorios encapsulan persistencia. EvidenceStorageService actúa como abstracción de almacenamiento y permite seleccionar PocketBase o almacenamiento local mediante configuración. Las integraciones externas se encapsulan para que sus errores no contaminen directamente la lógica de presentación.

Pruebas realizadas

Nivel	Prueba	Resultado
Backend	Maven test: controladores, contexto, Ollama y servicios de unidades.	12/12 aprobadas.
Frontend	Angular/Karma: componentes y servicios.	7/7 aprobadas.
Clúster	Estado de servicios y réplicas.	2/2 frontend, 2/2 backend; demás 1/1.
Salud	Frontend, backend, PocketBase, Ollama y ngrok.	HTTP 200 / backend UP.
Persistencia	Conteos y estados en PostgreSQL.	Datos consistentes y consultables.
IoT	WiFi, DNS, HTTPS, autenticación técnica y telemetría.	Flujo funcional con clave coincidente.

La base consultada durante la verificación contenía 6 reportes ciudadanos, 6 unidades, 2 evidencias multimedia, 1 orden Premium y 85 lecturas IoT. Cuatro reportes estaban ATENDIDOS y dos PENDIENTES; la orden registrada figuraba PAGADA. Estas cifras son una instantánea de prueba y pueden variar con nuevas ejecuciones.

APROBADO	Backend: 12 pruebas, sin fallos, errores ni omisiones.
-----------------	--

APROBADO	Frontend: 7 pruebas distribuidas en 4 archivos, todas exitosas.
-----------------	---

VERIFICADO	Servicios del stack operativos y endpoints de salud accesibles.
-------------------	---

Resultados obtenidos

El administrador recibe y gestiona reportes; el operador obtiene la asignación sin recargar gracias a eventos y reconciliación.

Solo se ofrecen para despacho unidades con operador activo y estado DISPONIBLE.

El cierre publica el resultado en tiempo real, libera la unidad y bloquea un nuevo despacho del reporte ATENDIDO.

Las evidencias se almacenan fuera de PostgreSQL y conservan hash SHA-256 y referencia relacional.

El pago Sandbox genera una orden Premium y programa la instalación después de la captura válida.

La telemetría real del ESP32 se incorpora al reporte con primera y última toma de aire.

Ollama responde dentro del dominio operacional y utiliza el estado del sistema como contexto.

Problemas encontrados y soluciones aplicadas

Problema	Causa	Solución aplicada / recomendada
Asignación no aparecía hasta recargar.	Eventos entre réplicas con broker simple.	WebSocket más reconciliación REST y eventos de cierre.
IoT mostraba conectado sin transmitir.	Se usaba la existencia de lecturas históricas.	Estado por antigüedad de la última telemetría.
HTTP 401 del ESP32.	Clave técnica distinta entre firmware y stack.	Unificar IOT_NODE_KEY y reiniciar el servicio.
HTTPS 421 / conexión rechazada.	Ruta/túnel/red y SNI.	Diagnóstico DNS/TLS, URL HTTPS y túnel correcto.
OAuth origin_mismatch.	Origen localhost no autorizado.	Registrar el origen exacto en Google Cloud.
Multimedia ausente.	Configuración/token PocketBase y referencias.	Proveedor PocketBase, colección, proxy y SHA-256.
Réplicas 0/2 temporalmente.	Base de datos no saludable o recursos.	Healthchecks, orden de arranque y revisión de tareas.

Matriz de cumplimiento del enunciado

Requisito	Estado	Observación verificable
Docker Swarm y réplicas	Cumple	Frontend 2, backend 2 y PostgreSQL 1.
REST síncrono	Cumple	46 endpoints mapeados aproximadamente.
Cola de mensajes externa	Parcial	Existe cola en memoria; falta RabbitMQ/Kafka durable.
Pagos Sandbox	parcial Cumple	Pago y conciliación listos; falta consola admin completa/auditoría de cambios.
OTP por correo	Cumple	HTML, expiración y ejecución @Async.
Recuperación y adjuntos	Pendiente	No fueron identificados en el código revisado.
TOTP	parcial Cumple	Alta y validación listas; falta desactivación autoservicio.
PocketBase privado	Cumple	Acceso vía backend, reglas bloqueadas, SHA-256.
Inteligencia artificial	Cumple	Ollama llama3.2 con dominio acotado.
Soft delete, paginación y filtros	Parcial	Estados conservan operaciones; faltan patrones uniformes en todos los listados.
README y .env.example	Cumple	Guías Docker y ejemplo sin secretos.

La matriz evita presentar como terminadas funciones que no aparecen en la implementación. Para alcanzar el 100 % del enunciado se debe incorporar un broker durable, recuperación de contraseña, adjuntos de correo, desactivación TOTP, consola administrativa de transacciones con auditoría y una estrategia uniforme de borrado lógico/paginación.

Conclusiones finales

La ampliación transformó el prototipo del segundo parcial en un ecosistema distribuido funcional. La plataforma coordina actores, datos relacionales, archivos, pagos, IA e IoT sin perder el ciclo operacional del incidente. El uso de réplicas demuestra autorrecuperación frente a fallos de tareas y el empleo de una sola base conserva una fuente de verdad coherente.

La integración ESP32 constituye el aporte más tangible: las proyecciones del informe anterior se materializaron en un dispositivo físico que mide el ambiente post-incendio y entrega sus datos al informe del operador. La comparación Wokwi-prototipo confirma una metodología incremental de simulación, construcción y validación.

La revisión también evidencia que distribuir no significa únicamente añadir contenedores. WebSocket con broker local, una cola volátil y un clúster de un solo nodo tienen límites claros.

Reconocerlos permite proponer una evolución responsable hacia RabbitMQ, relay STOMP, múltiples nodos físicos, observabilidad y respaldos coordinados.

Recomendaciones

Incorporar RabbitMQ como cola durable y como relay STOMP compartido entre réplicas.

Añadir un segundo nodo físico al Swarm para tolerar la pérdida del anfitrión.

Completar recuperación de contraseña, adjuntos HTML y desactivación TOTP protegida.

Crear una consola administrativa de pagos con filtros, auditoría, conciliación y reembolsos.

Uniformar borrado lógico, restauración, paginación y filtros en usuarios, pagos y archivos.

Agregar métricas Prometheus/Grafana, trazas y alertas de salud.

Calibrar el MQ-2 con condiciones controladas; no usar el prototipo como certificación profesional de habitabilidad.

Referencias bibliográficas ampliadas

Docker, Inc. (2026). How services work. <https://docs.docker.com/engine/swarm/how-swarm-mode-works/services/>

Docker, Inc. (2026). Use Swarm mode routing mesh. <https://docs.docker.com/engine/swarm/ingress/>

Spring. (2026). WebSocket STOMP overview. <https://docs.spring.io/spring-framework/reference/web/websocket/stomp/overview.html>

Spring. (2026). Task execution and scheduling. <https://docs.spring.io/spring-framework/reference/integration/scheduling.html>

PayPal. (2026). Orders API v2. <https://developer.paypal.com/docs/api/orders/v2/>

PocketBase. (2026). Files upload and handling. <https://pocketbase.io/docs/files-handling/>

PocketBase. (2026). API rules and filters. <https://pocketbase.io/docs/api-rules-and-filters/>

M'Raihi, D., Machani, S., Pei, M., & Rydell, J. (2011). TOTP: Time-Based One-Time Password Algorithm (RFC 6238). <https://datatracker.ietf.org/doc/html/rfc6238>

Fette, I., & Melnikov, A. (2011). The WebSocket Protocol (RFC 6455). <https://datatracker.ietf.org/doc/html/rfc6455>

Ollama. (2026). API documentation. <https://docs.ollama.com/api>

Angular. (2026). Angular documentation. <https://angular.dev/>

PostgreSQL Global Development Group. (2026). PostgreSQL documentation. <https://www.postgresql.org/docs/>

Anexos de la ampliación

Anexo A. Repositorio y archivos de ejecución

Repositorio: https://github.com/Joseph-Franco692/Proyecto_Emergencias_jf. Los archivos `docker-stack.yml`, `compose.yaml`, `DOCKER_SWARM.md`, `POCKETBASE_EVIDENCIAS.md`, `.env.docker.example` y los scripts de la carpeta `scripts` documentan el despliegue. El archivo `.env.docker` real se mantiene excluido de Git.

Anexo B. Comandos de verificación

El Algoritmo 13 reúne la evidencia reproducible de clúster, salud y pruebas. Ningún comando imprime variables sensibles ni el contenido del archivo de entorno.

Algoritmo 13. *Verificación integral sin exponer secretos.*

```
# Servicios y tareas
docker service ls
docker stack services gestion-bomberil
docker stack ps gestion-bomberil

# Salud
```

```

Invoke-WebRequest http://localhost:8082/api/health
Invoke-WebRequest http://localhost:8091/api/health
Invoke-WebRequest http://localhost:11435/api/tags

```

```

# Pruebas
cd emergencias
mvn test
cd ../central-bomberos
npm test -- --watch=false

```

Anexo C. Correspondencia de pines del prototipo IoT

Elemento	Pin ESP32	Función
MQ-2 AOUT	GPIO 34	Entrada analógica de humo/gas.
Buzzer	GPIO 23	Alarma acústica.
Pulsador	GPIO 4	Encendido/apagado lógico.
RGB rojo	GPIO 5	Estado crítico.
RGB verde	GPIO 19	Estado respirable.
RGB azul	GPIO 18	Sistema en espera/apagado.
LCD SDA	GPIO 21	Bus I2C de datos.
LCD SCL	GPIO 22	Bus I2C de reloj.

Los valores de umbral son parámetros de calibración del prototipo y deben ajustarse con mediciones de referencia. El dispositivo es una demostración académica; no reemplaza instrumentos certificados ni protocolos profesionales para declarar habitable una estructura.